

Aug 5,2026: Python Practice Problems: Graphs, Heap, Priority Queue

Student Problem Mapping

Student	Problem 1	Problem 2
Aswini S	# P01 - Graph Basics	# P02 - Graph Basics
Bhuvanesh B	# P03 - Graph Basics	# P04 - Graph Basics
Chandran S	# P05 - BFS	# P06 - BFS
Dharun M	# P07 - BFS	# P08 - BFS
Dinesh Babu N	# P09 - BFS	# P10 - DFS
Gokul S	# P11 - DFS	# P12 - DFS
Gokula Kannan M	# P13 - DFS	# P14 - DFS
Hashwanth H	# P15 - Heap	# P16 - Heap
Hema prabha R S	# P17 - Heap	# P18 - Heap
Kabilan S	# P19 - Heap	# P20 - Priority Queue
Manikandan M	# P21 - Priority Queue	# P22 - Priority Queue
Naveen S	# P23 - Priority Queue	# P24 - Dijkstra
Praveenraj A	# P25 - Dijkstra	# P26 - Dijkstra
Rooban Kumar S	# P27 - Dijkstra	# P28 - Bellman-Ford
Sanjai S	# P29 - Bellman-Ford	# P30 - Bellman-Ford
SENTHILNATHAN P	# P31 - Topological Sorting	# P32 - Topological Sorting
SUJITHASRI G	# P33 - Topological Sorting	# P34 - Prim's Algorithm
Yogeshwaran E	# P35 - Prim's Algorithm	# P36 - Kruskal's Algorithm

P01 – Graph Basics: Degree of Each Node

Graph Diagram

```
A ---- B
|      |
C      D
```

Problem Statement

Write a Python program to read an undirected graph and print the degree of each node.

Input Format

- First line contains two integers n and m.
- Second line contains n node names.
- Next m lines contain two node names representing an undirected edge.

Output Format

- Print each node and its degree in the same order as input.

Constraints

$$0 \leq m \leq 50$$

Node names are strings without spaces.

Sample Input

```
4 3
A B C D
A B
A C
B D
```

Sample Output

```
A 2
B 2
```

C 1

D 1

Explanation

- Node A is connected to B and C, so its degree is 2.

P02 – Graph Basics: Print Adjacency List

Graph Diagram

```
A ---- B ---- D
|
C
```

Problem Statement

Write a Python program to store an undirected graph using an adjacency list and print the neighbors of each node.

Input Format

- First line contains n and m .
- Second line contains n node names.
- Next m lines contain two connected nodes.

Output Format

- Print each node followed by its sorted neighbor list.

Constraints

$1 \leq n \leq 20$

$0 \leq m \leq 50$

Sample Input

```
4 3
A B C D
```

A B

A C

B D

Sample Output

A: B C

B: A D

C: A

D: B

Explanation

- The adjacency list stores all direct neighbors of each node.

P03 – Graph Basics: Directed Adjacency Matrix

Graph Diagram

A ---> B

| |

v v

C ---> D

Problem Statement

- Write a Python program to create and print the adjacency matrix of a directed graph.

Input Format

- First line contains n and m.
- Second line contains n node names.
- Next m lines contain directed edges u v.

Output Format

- Print the adjacency matrix with 1 if an edge exists, otherwise 0.

Constraints

$1 \leq n \leq 10$

$0 \leq m \leq 30$

Sample Input

```
4 4
A B C D
A B
A C
B D
C D
```

Sample Output

```
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
```

Explanation

- There is an edge from A to B, so row A, column B is 1.

P04 – Graph Basics: Check Edge Existence

Graph Diagram

```
A ---- B
|
C
```

Problem Statement

- Write a Python program to check whether an edge exists between two nodes in an undirected graph.

Input Format

- First line contains n and m .
- Second line contains node names.
- Next m lines contain undirected edges.
- Next line contains integer q .
- Next q lines contain two nodes to query.

Output Format

- Print YES if an edge exists, otherwise print NO.

Constraints

$1 \leq n \leq 20$

$1 \leq q \leq 20$

Sample Input

```
3 2
A B C
A B
A C
3
A B
B C
A C
```

Sample Output

```
YES
NO
YES
```

Explanation

- B and C are not directly connected.

P05 – BFS: Breadth First Traversal

Graph Diagram

```
A ---- B ---- D
|
C ---- E
```

Problem Statement

Write a Python program to perform BFS traversal from a given starting node.

Input Format

- First line contains n and m.
- Second line contains node names.
- Next m lines contain undirected edges.
- Last line contains the starting node.

Output Format

- Print BFS traversal order.

Constraints

$1 \leq n \leq 20$

Visit neighbors in alphabetical order.

Sample Input

```
5 4
A B C D E
A B
A C
B D
C E
A
```

Sample Output

```
A B C D E
```

Explanation

BFS visits nodes level by level starting from A.

P06 – BFS: Shortest Distance in Unweighted Graph

Graph Diagram

```
A ---- B ---- D
|               |
C ----- E
```

Problem Statement

- Write a Python program to find the shortest number of edges from a source node to a destination node using BFS.

Input Format

- First line contains n and m .
- Second line contains node names.
- Next m lines contain undirected edges.
- Last line contains source and destination.

Output Format

- Print the shortest distance.

Constraints

$1 \leq n \leq 20$

If no path exists, print `-1`.

Sample Input

```
5 5
A B C D E
A B
A C
B D
```

D E
C E
A E

Sample Output

2

Explanation

The shortest path is A -> C -> E.

P07 – BFS: Print Level of Each Node

Graph Diagram

```
A
| \
B  C
|  \
D   E
```

Problem Statement

Write a Python program to print the BFS level of each reachable node from the starting node.

Input Format

- First line contains n and m.
- Second line contains node names.
- Next m lines contain edges.
- Last line contains starting node.

Output Format

Print each node and its level in BFS order.

Constraints

$1 \leq n \leq 20$

Sample Input

```
5 4
A B C D E
A B
A C
B D
C E
A
```

Sample Output

```
A 0
B 1
C 1
D 2
E 2
```

Explanation

- Nodes B and C are one edge away from A.

P08 – BFS: Count Connected Components

Graph Diagram

A ---- B C ---- D E

Problem Statement

Write a Python program to count the number of connected components in an undirected graph using BFS.

Input Format

First line contains n and m .

Second line contains node names.

Next m lines contain edges.

Output Format

Print the number of connected components.

Constraints

$1 \leq n \leq 30$

Sample Input

```
5 2
A B C D E
A B
C D
```

Sample Output

```
3
```

Explanation

The components are {A,B}, {C,D}, and {E}.

P09 – BFS: Check Path Exists

Graph Diagram

```
A ---- B ---- C      D
```

Problem Statement

Write a Python program to check whether a path exists between two nodes using BFS.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain edges.

Last line contains source and destination.

Output Format

Print PATH EXISTS or NO PATH.

Constraints

$1 \leq n \leq 20$

Sample Input

```
4 2
A B C D
A B
B C
A D
```

Sample Output

```
NO PATH
```

Explanation

Node D is disconnected from A.

P10 – DFS: Depth First Traversal

Graph Diagram

```
A ---- B
|      |
C ---- D
```

Problem Statement

Write a Python program to perform DFS traversal from a given starting node.

Input Format

First line contains n and m .

Second line contains node names.

Next m lines contain undirected edges.

Last line contains starting node.

Output Format

Print DFS traversal order.

Constraints

$1 \leq n \leq 20$

Visit neighbors in alphabetical order.

Sample Input

```
4 4
A B C D
A B
A C
B D
C D
A
```

Sample Output

```
A B D C
```

Explanation

DFS goes as deep as possible before backtracking.

P11 – DFS: Count Reachable Nodes

Graph Diagram

A ---- B ---- C D

Problem Statement

Write a Python program to count how many nodes are reachable from a given starting node using DFS.

Input Format

First line contains n and m .

Second line contains node names.

Next m lines contain edges.

Last line contains starting node.

Output Format

Print the number of reachable nodes.

Constraints

$1 \leq n \leq 20$

Sample Input

```
4 2
A B C D
A B
B C
A
```

Sample Output

```
3
```

Explanation

From A, nodes A, B, and C are reachable.

P12 – DFS: Detect Cycle in Undirected Graph

Graph Diagram

A ---- B
| |
C ---- D

Problem Statement

Write a Python program to detect whether an undirected graph contains a cycle using DFS.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain edges.

Output Format

Print CYCLE FOUND or NO CYCLE.

Constraints

$1 \leq n \leq 20$

Sample Input

```
4 4
A B C D
A B
B D
D C
C A
```

Sample Output

```
CYCLE FOUND
```

Explanation

The graph contains the cycle A -> B -> D -> C -> A.

P13 – DFS: Count Components

Graph Diagram

A ---- B C D ---- E

Problem Statement

Write a Python program to count connected components using DFS.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain edges.

Output Format

Print component count.

Constraints

$1 \leq n \leq 30$

Sample Input

```
5 2
A B C D E
A B
D E
```

Sample Output

```
3
```

Explanation

The components are {A,B}, {C}, and {D,E}.

P14 – DFS: Print One Path Between Two Nodes

Graph Diagram

```
A ---- B ---- D
|
C ---- E
```

Problem Statement

Write a Python program to print one path from source to destination using DFS.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain edges.

Last line contains source and destination.

Output Format

Print the path. If no path exists, print NO PATH.

Constraints

$1 \leq n \leq 20$

Visit neighbors in alphabetical order.

Sample Input

```
5 4
A B C D E
A B
A C
B D
C E
A D
```

Sample Output

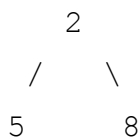
```
A B D
```

Explanation

DFS finds the path from A to D.

P15 – Heap: Min Heap Operations

Heap Diagram



Problem Statement

Write a Python program to perform min heap operations. For every extract, print the removed minimum value.

Input Format

First line contains integer q .

Next q lines contain either:

insert x

extract

Output Format

Print extracted values, one per line.

Constraints

$1 \leq q \leq 50$

$1 \leq x \leq 1000$

Sample Input

```
5
insert 5
insert 2
insert 8
```

extract

extract

Sample Output

2

5

Explanation

The minimum values are removed first.

P16 – Heap: Kth Smallest Number

Heap Diagram

Numbers: 7 3 9 1 5

Min Heap helps remove smallest values first.

Problem Statement

Write a Python program to find the kth smallest number using a heap.

Input Format

First line contains n.

Second line contains n integers.

Third line contains k.

Output Format

Print the kth smallest number.

Constraints

$1 \leq k \leq n \leq 50$

Sample Input

5

7 3 9 1 5

3

Sample Output

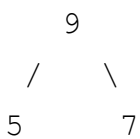
5

Explanation

Sorted order is 1 3 5 7 9; the 3rd smallest is 5.

P17 – Heap: Max Heap Root

Heap Diagram



Problem Statement

Write a Python program to build a max heap and print the maximum element.

Input Format

First line contains n .

Second line contains n integers.

Output Format

Print the maximum element.

Constraints

$1 \leq n \leq 50$

Sample Input

5
4 9 2 7 5

Sample Output

Explanation

In a max heap, the largest value is at the root.

P18 – Heap: Running Minimum

Heap Diagram

Insert numbers one by one:

4, 2, 7, 1

Current minimum changes after each insertion.

Problem Statement

Write a Python program to print the minimum value after each insertion into a min heap.

Input Format

First line contains n .

Second line contains n integers.

Output Format

Print running minimum after every insertion.

Constraints

$1 \leq n \leq 50$

Sample Input

4
4 2 7 1

Sample Output

4 2 2 1

Explanation

After inserting 1, the minimum becomes 1.

P19 – Heap: Heap Sort Ascending

Heap Diagram

Min Heap removes elements in ascending order.

Problem Statement

Write a Python program to sort numbers in ascending order using a min heap.

Input Format

First line contains n .

Second line contains n integers.

Output Format

Print sorted numbers.

Constraints

$1 \leq n \leq 50$

Sample Input

```
6
10 3 6 1 8 2
```

Sample Output

```
1 2 3 6 8 10
```

Explanation

Repeatedly removing from min heap gives ascending order.

P20 – Priority Queue: Task Scheduling

Diagram

Task priority:

Study(1), Play(3), Eat(2)

Lower number means higher priority.

Problem Statement

Write a Python program to process tasks using a priority queue. Smaller priority value should be processed first.

Input Format

First line contains n.

Next n lines contain task name and priority.

Output Format

Print task names in processing order.

Constraints

$1 \leq n \leq 50$

If priorities are same, process lexicographically smaller task first.

Sample Input

```
3
Study 1
Play 3
Eat 2
```

Sample Output

```
Study Eat Play
```

Explanation

Priority 1 is processed before 2 and 3.

P21 – Priority Queue: Hospital Queue

Diagram

Patient severity:

Bob (5), Ana (8), Tom (6)

Higher severity is treated first.

Problem Statement

Write a Python program to process patients in a hospital queue. Higher severity should be treated first.

Input Format

First line contains n .

Next n lines contain patient name and severity.

Output Format

Print patient names in treatment order.

Constraints

$1 \leq n \leq 50$

If severity is same, earlier input comes first.

Sample Input

```
3
Bob 5
Ana 8
Tom 6
```

Sample Output

```
Ana Tom Bob
```

Explanation

Ana has the highest severity.

P22 – Priority Queue: Merge Sorted Lists

Diagram

List1: 1 4

List2: 2 5

List3: 3 6

Problem Statement

Write a Python program to merge multiple sorted lists using a priority queue.

Input Format

First line contains k , the number of lists.

For each list:

First value is list size.

Remaining values are sorted integers.

Output Format

Print the merged sorted list.

Constraints

$1 \leq k \leq 10$

Total numbers ≤ 100

Sample Input

3

2 1 4

2 2 5

2 3 6

Sample Output

1 2 3 4 5 6

Explanation

All sorted lists are merged into one sorted list.

P23 – Priority Queue: Top K Frequent Words

Diagram

Words:

apple banana apple mango banana apple

Problem Statement

Write a Python program to print the top k most frequent words using a priority queue.

Input Format

First line contains n.

Second line contains n words.

Third line contains k.

Output Format

Print top k words by frequency.

Constraints

$1 \leq k \leq n \leq 100$

If frequency is same, print lexicographically smaller word first.

Sample Input

```
6
apple banana apple mango banana apple
2
```

Sample Output

```
apple banana
```

Explanation

apple appears 3 times and banana appears 2 times.

P24 - Dijkstra Algorithm: Shortest Distance from Source

Graph Diagram

```
A --1-- B
|         |
4         2
|         |
C --1-- D
```

Problem Statement

Write a Python program to find the shortest distance from a source node to all other nodes using Dijkstra's algorithm.

Input Format

First line contains n and m .

Second line contains node names.

Next m lines contain $u\ v\ w$.

Last line contains source node.

Output Format

Print each node and shortest distance from source.

Constraints

$1 \leq n \leq 20$

Edge weights are positive.

Sample Input

4 4
A B C D
A B 1
A C 4
B C 2
C D 1
A

Sample Output

A 0
B 1
C 3
D 4

Explanation

Shortest path to D is A -> B -> C -> D.

P25 - Dijkstra Algorithm: Shortest Path

Graph Diagram

```
A --2-- B --2-- D --1-- E
 \      |
  5      1
  \      |
    C --3-- E
```

Problem Statement

Write a Python program to print the shortest path and cost from source to destination using Dijkstra's algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain weighted undirected edges.

Last line contains source and destination.

Output Format

Print the path in one line and cost in the next line.

Constraints

$1 \leq n \leq 20$

Weights are positive.

Sample Input

```
5 6
A B C D E
A B 2
A C 5
B C 1
B D 2
C E 3
D E 1
A E
```

Sample Output

```
A B D E
5
```

Explanation

The shortest path from A to E has cost 5.

P26 – Dijkstra Algorithm: Nodes Within Distance K

Graph Diagram

```

A  --2--  B  --2--  C
|
5          |
|          |
D  -----  E

```

Problem Statement

Write a Python program to count how many nodes can be reached from a source within distance K.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain weighted edges.

Last line contains source and K.

Output Format

Print the count of reachable nodes within distance K, including the source.

Constraints

$1 \leq n \leq 20$

Weights are positive.

Sample Input

```

5 5
A B C D E
A B 2
B C 2
C E 1
A D 5
D E 1
A 4

```

Sample Output

```

3

```

Explanation

Nodes A, B, and C are within distance 4.

P27 – Dijkstra Algorithm: Network Delay Time

Graph Diagram

```
A  --1--  B  --2--  C
 \                |
  \-----4-----D
```

Problem Statement

Write a Python program to find the time needed for a signal from source to reach all nodes using Dijkstra's algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain directed weighted edges u v w.

Last line contains source node.

Output Format

Print the maximum shortest distance. If any node is unreachable, print -1.

Constraints

$1 \leq n \leq 20$

Weights are positive.

Sample Input

```
4 3
A B C D
A B 1
B C 2
```

A D 4

A

Sample Output

4

Explanation

The farthest reachable node from A is D with distance 4.

P28 – Bellman-Ford Algorithm: Shortest Distance with Negative Edge

Graph Diagram

```
A --4--> B
|         |
5         -2
|         v
v         C --3--> D
C
```

Problem Statement

Write a Python program to find shortest distances from a source using Bellman-Ford algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain directed edges u v w.

Last line contains source node.

Output Format

Print shortest distance to each node.

Constraints

$1 \leq n \leq 20$

Edge weights may be negative.

No negative cycle in ##### Sample Input.

Sample Input

```
4 4
A B C D
A B 4
A C 5
B C -2
C D 3
A
```

Sample Output

```
A 0
B 4
C 2
D 5
```

Explanation

Even with a negative edge $B \rightarrow C$, Bellman-Ford finds correct distances.

P29 – Bellman-Ford Algorithm: Detect Negative Cycle

Graph Diagram

```
A --1--> B
^         |
|         -2
|         v
+--- -2 ---C
```

Problem Statement

Write a Python program to detect whether a directed graph contains a negative weight cycle.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain directed weighted edges.

Last line contains source node.

Output Format

Print NEGATIVE CYCLE or NO NEGATIVE CYCLE.

Constraints

$1 \leq n \leq 20$

Sample Input

```
3 3
A B C
A B 1
B C -2
C A -2
A
```

Sample Output

```
NEGATIVE CYCLE
```

Explanation

The cycle $A \rightarrow B \rightarrow C \rightarrow A$ has total weight -3 .

P30 – Bellman–Ford Algorithm: Cheapest Route

Graph Diagram

```
S --6--> A --2--> T
S --3--> B --4--> T
B --1--> A
```

Problem Statement

Write a Python program to find the cheapest cost from source to destination using Bellman-Ford algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain directed weighted edges.

Last line contains source and destination.

Output Format

Print the cheapest cost. If destination is unreachable, print -1.

Constraints

$1 \leq n \leq 20$

Edge weights may be negative, but no negative cycle.

Sample Input

```
4 5
S A B T
S A 6
A T 2
S B 3
B T 4
B A 1
S T
```

Sample Output

Explanation

The cheapest route is $S \rightarrow B \rightarrow A \rightarrow T$ with cost $3 + 1 + 2 = 6$.

P31 – Topological Sorting: Course Order

Graph Diagram

```
Math ---> DSA ---> Project
|
v
Python
```

Problem Statement

Write a Python program to print a valid course completion order using topological sorting.

Input Format

First line contains n and m .

Second line contains course names.

Next m lines contain prerequisite relation $u\ v$, meaning u must be completed before v .

Output Format

Print one valid topological order.

Constraints

$1 \leq n \leq 20$

Graph is a DAG.

Sample Input

4 3

Math Python DSA Project

Math Python

Sample Output

Math DSA Project Python

Explanation

Math must come before both Python and DSA.

P32 – Topological Sorting: Check Task Completion Possible

Graph Diagram

A ---> B ---> C

Problem Statement

Write a Python program to check whether all tasks can be completed. If the directed graph has a cycle, tasks cannot be completed.

Input Format

First line contains n and m.

Second line contains task names.

Next m lines contain dependency u v.

Output Format

Print POSSIBLE or NOT POSSIBLE.

Constraints

$1 \leq n \leq 20$

Sample Input

3 2

A B C

A B

B C

Sample Output

POSSIBLE

Explanation

There is no cycle, so tasks can be completed.

P33 – Topological Sorting: Count Starting Tasks

Graph Diagram

A ---> C

B ---> C

D

Problem Statement

Write a Python program to count how many tasks have no prerequisites.

Input Format

First line contains n and m.

Second line contains task names.

Next m lines contain dependency u v.

Output Format

Print the count of tasks with indegree 0.

Constraints

$1 \leq n \leq 20$

Sample Input

```
4 2
A B C D
A C
B C
```

Sample Output

```
3
```

Explanation

Tasks A, B, and D have no prerequisites.

P34 – Prim's Algorithm: Minimum Spanning Tree Weight

Graph Diagram

```
A  --1--  B
|          / |
3         2  4
|          / |
C  --5---  D
```

Problem Statement

Write a Python program to find the total weight of the Minimum Spanning Tree using Prim's algorithm.

Input Format

First line contains n and m .

Second line contains node names.

Next m lines contain weighted undirected edges.

Output Format

Print total MST weight.

Constraints

$1 \leq n \leq 20$

Graph is connected.

Weights are positive.

Sample Input

```
4 5
A B C D
A B 1
A C 3
B C 2
B D 4
C D 5
```

Sample Output

```
7
```

Explanation

MST edges can be A-B, B-C, and B-D, total weight $1 + 2 + 4 = 7$.

P35 – Prim's Algorithm: Print MST Edges

Graph Diagram

```
A  --2--  B
|          |
1          3
|          |
C  --4--  D
```

Problem Statement

Write a Python program to print the edges selected in the Minimum Spanning Tree using Prim's algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain weighted undirected edges.

Last line contains starting node.

Output Format

Print MST edges and total weight.

Constraints

$1 \leq n \leq 20$

Graph is connected.

Sample Input

```
4 4
A B C D
A B 2
A C 1
B D 3
C D 4
A
```

Sample Output

```
A C 1
A B 2
B D 3
Total 6
```

Explanation

Prim's algorithm chooses the smallest edge that connects a new node.

P36 – Kruskal's Algorithm: Minimum Spanning Tree

Graph Diagram

```
A  --1--  B
|          |
4          2
|          |
C  --3--  D
```

Problem Statement

Write a Python program to find the Minimum Spanning Tree using Kruskal's algorithm.

Input Format

First line contains n and m.

Second line contains node names.

Next m lines contain weighted undirected edges.

Output Format

Print selected MST edges in increasing weight order and print total weight.

Constraints

$1 \leq n \leq 20$

Graph is connected.

Weights are positive.

Sample Input

```
4 4
A B C D
A B 1
B D 2
```

C D 3

A C 4

Sample Output

A B 1

B D 2

C D 3

Total 6

Explanation

Kruskal's algorithm sorts edges by weight and adds edges that do not form a cycle.